

Proyecto Integrador Final: Arquitectura de Software II

Guía de Escenarios, Requerimientos Técnicos y Rúbrica de Calificación (v9)

1. Instrucciones Generales

El objetivo es diseñar e implementar una arquitectura de microservicios que resuelva una problemática de negocio real. Se debe priorizar el pensamiento crítico y la justificación técnica de cada decisión (trade-offs).

2. Escenarios de Proyecto (Casos de Estudio)

1. AgroSmart: El Futuro del Campo Orgánico

Contexto: La exportadora "Tierra Viva" ha perdido el 20% de su cosecha debido a riegos innecesarios que pudren la raíz y falta de fertilización a tiempo. Actualmente, los agrónomos toman decisiones basadas en la "intuición" y visitas semanales. La empresa quiere un sistema de Smart Farming que tome decisiones autónomas basadas en telemetría de suelo y datos climáticos en tiempo real.

El Problema: El negocio necesita automatizar el riego, pero no puede depender de una base de datos lenta para decidir si abre o no una válvula. Además, cada gota de agua debe ser auditada para certificaciones internacionales.

Reglas de Negocio:

- El sistema debe denegar el riego automático si la humedad del suelo es $> 25\%$ o si la API climática (síncrona) indica que lloverá en la zona en menos de 4 horas.
- Todo evento de riego debe quedar registrado de forma asíncrona para auditoría; si el log falla, el riego no se detiene, pero se debe generar una alerta de "Fallo de Trazabilidad".
- Un lote de cultivo entra en estado de "Alerta Sanitaria" si el sensor de pH reporta valores fuera del rango (5.5-7.0) por más de 3 lecturas consecutivas.

Arquitectura Sugerida:

- MS Principal: **MS-Crops** (Hexagonal / PostgreSQL)
- MS Síncrono: **MS-SensorProxy** (REST / Redis)
- MS Asíncrono: **MS-IrrigationLogs** (Broker / MongoDB)

2. VoltNet: Gestión de Carga Eléctrica Urbana

Contexto: La ciudad está instalando cargadores para vehículos eléctricos (EV). "VoltNet" es la empresa encargada de gestionar los puntos de carga. El problema es que si todos los autos cargan al mismo tiempo a máxima potencia, la red eléctrica del barrio colapsa.

El Problema: Se requiere un orquestador que valide el estado de la estación, autorice al usuario y registre el consumo para cobro posterior sin bloquear al usuario en la estación.

Reglas de Negocio:

- *No se puede iniciar una carga si el estado actual de la estación (verificado síncronamente) indica una carga total de red > 100kW.*
- *El sistema debe verificar que el usuario tenga un método de pago activo; si tiene deudas vencidas de más de 30 días, el inicio de carga se bloquea automáticamente.*
- *Al finalizar, el cálculo de kW consumidos se envía a un sistema de facturación asíncrono. La sesión se cierra exitosamente aunque el sistema de facturación esté fuera de línea.*

Arquitectura Sugerida:

- MS Principal: **MS-ChargeOrchestrator** (Hexagonal / MySQL)
- MS Síncrono: **MS-GridLoad** (REST / Redis)
- MS Asíncrono: **MS-Billing** (Broker / PostgreSQL)

3. BetFlow: Apuestas en Vivo "In-Game"

Contexto: "BetFlow" es una casa de apuestas que permite apostar durante los partidos. Su mayor pérdida ocurre por el "Latencia de Ventaja": personas que apuestan cuando ya saben que hubo un gol, pero el sistema aún no ha actualizado la cuota.

El Problema: El sistema debe validar el saldo del usuario al instante (síncrono) pero actualizar las cuotas y cerrar mercados de forma masiva ante eventos (asíncrono) para evitar fraudes.

Reglas de Negocio:

- *Una apuesta se rechaza si se recibe después de un evento "Grave" (Gol, Roja) procesado asíncronamente, incluso si la transacción llegó al servidor segundos después.*
- *El saldo del usuario debe congelarse síncronamente al momento de la apuesta; no se permiten apuestas si el saldo disponible es inferior al monto solicitado.*
- *El límite máximo de apuesta por usuario en un solo evento es de \$500 USD; cualquier intento superior debe ser bloqueado por el dominio.*

Arquitectura Sugerida:

- MS Principal: **MS-BettingCore** (Hexagonal / PostgreSQL)
- MS Síncrono: **MS-Wallet** (REST / MySQL)
- MS Asíncrono: **MS-OddsUpdater** (Broker / Redis)

4. BioLibrary: Préstamo Digital de Alto Costo

Contexto: Una red de universidades comparte libros digitales cuyos derechos de autor cuestan miles de dólares. Las licencias son limitadas (ej. solo 5 personas pueden leer el mismo libro a la vez).

El Problema: El sistema actual permite que estudiantes con mal rendimiento académico o sanciones acaparen libros que otros necesitan con urgencia. Se necesita un control estricto de acceso y revocación de licencias.

Reglas de Negocio:

- *Un préstamo solo es válido si el sistema académico (síncrono) confirma que el estudiante está matriculado y no tiene sanciones vigentes.*
- *Si un estudiante tiene un promedio acumulado < 3.2 , el sistema solo le permite tener 1 libro activo a la vez.*
- *La licencia digital expira automáticamente si el sistema de logs asíncronos detecta que el libro no ha tenido actividad (scroll/lectura) en los últimos 30 minutos.*

Arquitectura Sugerida:

- MS Principal: **MS-DigitalCatalog** (Hexagonal / MongoDB)
- MS Síncrono: **MS-StudentValidator** (REST / SQL Server)
- MS Asíncrono: **MS-ActivityMonitor** (Broker / PostgreSQL)

5. MatchPoint: Reservas de Padel y Ranking

Contexto: El club "MatchPoint" sufre por usuarios que reservan canchas y no asisten (No-Show), quitándole el espacio a jugadores competitivos. Quieren un sistema que no solo reserve, sino que gestione la reputación del jugador.

El Problema: El flujo de reserva debe ser rápido, pero la actualización del ranking y las penalizaciones por inasistencia deben ser procesos robustos que no afecten la navegación del usuario.

Reglas de Negocio:

- *Para reservar en horario "Premium" (18:00-22:00), el usuario debe tener una membresía activa verificada síncronamente.*
- *Si un usuario cancela una reserva con menos de 2 horas de antelación, el sistema asíncrono debe generar una penalización de "Baja Confiabilidad" que le impedirá reservar en horario Premium por 1 semana.*
- *No se puede crear una reserva para un partido "Ranked" si los jugadores invitados tienen una diferencia de nivel > 2.0 puntos entre ellos.*

Arquitectura Sugerida:

- MS Principal: **MS-BookingManager** (Hexagonal / PostgreSQL)
- MS Síncrono: **MS-Identity** (REST / MySQL)
- MS Asíncrono: **MS-PenaltyRank** (Broker / MongoDB)

6. EcoMonitor: Red de Alerta Ambiental

Contexto: El gobierno local ha desplegado sensores de CO2. El sistema debe procesar miles de datos para declarar "Emergencia Ambiental" y restringir el tráfico vehicular.

El Problema: Un sensor fallido puede dar lecturas falsas. El sistema debe analizar la consistencia de los datos (síncrono) y disparar alertas masivas a la población (asíncrono).

Reglas de Negocio:

- Una alerta de "Emergencia" solo se valida si el promedio de 3 sensores diferentes en la misma zona geográfica supera los 150 $\mu\text{g}/\text{m}^3$.
- El sistema debe consultar síncronamente el mapa de "Zonas Sensibles" (hospitales, escuelas) para elevar la prioridad de la alerta si el foco de contaminación está cerca.
- Si un sensor envía datos inconsistentes (ej. saltos de 20 a 500 en 1 segundo), el dato se ignora y se envía un evento asíncrono para mantenimiento técnico.

Arquitectura Sugerida:

- MS Principal: **MS-PollutionAnalytics** (Hexagonal / TimescaleDB)
- MS Síncrono: **MS-ZoneService** (REST / PostgreSQL)
- MS Asíncrono: **MS-EmergencyAlert** (Broker / Redis)

7. PartFinder: El Marketplace de Repuestos

Contexto: "PartFinder" conecta talleres mecánicos con cientos de bodegas de repuestos. Los talleres necesitan saber el stock real para no dejar un auto desarmado esperando una pieza que no llegará.

El Problema: Las bodegas tienen sistemas viejos y lentos. El sistema principal debe orquestar la búsqueda rápida y manejar el historial de búsquedas para sugerir stock a los proveedores.

Reglas de Negocio:

- Al buscar un repuesto, el sistema debe consultar síncronamente al proveedor; si este tarda más de 800ms en responder, el sistema debe marcar el repuesto como "Disponibilidad Incierta".
- No se puede procesar un pedido si el taller tiene un cupo de crédito excedido en el microservicio financiero.
- Cada búsqueda fallida (repuesto no encontrado) debe notificarse asíncronamente a los proveedores para que estos ajusten su inventario futuro.

Arquitectura Sugerida:

- MS Principal: **MS-Aggregator** (Hexagonal / Elasticsearch)
- MS Síncrono: **MS-InventoryDirect** (REST / MySQL)
- MS Asíncrono: **MS-TrendCollector** (Broker / PostgreSQL)

8. Medipass: Agendamiento Especializado

Contexto: Una red de salud de alta complejidad gestiona citas para especialistas (Oncólogos, Cardiólogos). El error humano en el agendamiento cuesta vidas y recursos médicos.

El Problema: Se debe validar el seguro del paciente y la disponibilidad del médico en tiempo real, mientras se actualiza el historial clínico sin demorar la llamada de la cita.

Reglas de Negocio:

- *No se puede confirmar una cita si la aseguradora (vía proxy/externo) devuelve un código de "Procedimiento No Cubierto".*
- *El sistema debe bloquear la agenda del médico sincronamente para evitar el Double-Booking (dos personas a la misma hora).*
- *Tras confirmar la cita, el resumen de la misma debe enviarse al Historial Clínico Digital de forma asíncrona; el proceso de agenda no debe esperar a que el historial responda.*

Arquitectura Sugerida:

- MS Principal: **MS-AgendaHub** (Hexagonal / PostgreSQL)
- MS Síncrono: **MS-Insurance** (REST / Oracle)
- MS Asíncrono: **MS-EHRLogger** (Broker / MongoDB)

9. SmartLogistics: Gestión de Almacenes Autónomos

Contexto: Un centro de distribución masivo utiliza robots autónomos para el "picking" de productos. Se requiere optimizar el movimiento de los robots para reducir tiempos y evitar colisiones en los pasillos de alta densidad.

El Problema: El sistema debe coordinar las órdenes de pedido en tiempo real, validando la disponibilidad de los robots y su estado de carga, mientras se registra cada movimiento para el análisis de eficiencia logística.

Reglas de Negocio:

- *No se puede asignar una orden de despacho a un robot si su nivel de batería (verificado sincronamente) es inferior al 15%.*
- *Los productos marcados como "Frágiles" deben ser transportados a una velocidad reducida, configurada dinámicamente en el microservicio principal.*
- *Cada ruta completada debe enviarse asíncronamente a un sistema de analítica para generar mapas de calor de congestión en el almacén.*

Arquitectura Sugerida:

- MS Principal: **MS-WarehouseCore** (Hexagonal / PostgreSQL)
- MS Síncrono: **MS-RobotStatus** (REST / Redis)
- MS Asíncrono: **MS-LogisticsAnalytics** (Broker / MongoDB)

10. SkyWatch: Control de Tráfico de Drones Urbanos

Contexto: Una empresa de mensajería aérea opera una flota de drones en zonas metropolitanas. La coordinación del espacio aéreo es crítica para evitar accidentes y cumplir con las regulaciones de seguridad.

El Problema: Se requiere una plataforma que autorice planes de vuelo basándose en las condiciones meteorológicas y la congestión del tráfico, asegurando el registro de pruebas de entrega.

Reglas de Negocio:

- Un plan de vuelo solo se autoriza si el monitor meteorológico (síncrono) confirma que la velocidad del viento en la ruta es inferior a 50 km/h.
- El dron debe mantener una altitud mínima de seguridad según la zona, validando sincronamente los límites geográficos prohibidos (No-Fly Zones).
- Tras la entrega, la foto de evidencia capturada debe procesarse asíncronamente para notificar al cliente sin retrasar el regreso del dron a la base.

Arquitectura Sugerida:

- MS Principal: **MS-TrafficController** (Hexagonal / TimescaleDB)
- MS Síncrono: **MS-WeatherMonitor** (REST / Redis)
- MS Asíncrono: **MS-DeliveryProof** (Broker / MongoDB)

3. Rúbrica de Calificación

Componente	Descripción	Peso
SUSTENTACIÓN ORAL	Defensa de decisiones técnicas, análisis de trade-offs y pensamiento crítico.	40%
Arquitectura Hexagonal + SOLID	Separación de capas de dominio, puertos/adaptadores y aplicación de patrones GoF.	15%
Comunicación Sync/Async	Uso funcional de REST (Feign/WebClient) y Mensajería (Rabbit/Kafka).	10%
RFC y Modelo C4 (L2)	Documentación técnica justificando el diseño y diagrama de contenedores.	15%
Infraestructura Docker	Compose funcional con persistencia políglota (SQL + NoSQL).	10%
Observabilidad	Dashboards de Grafana y Trazabilidad en Jaeger funcionando.	10%
BONIFICACIONES	Proxy (Nginx) y/o Interfaz UI funcional consumiendo el MS Principal.	+1.0

4. Plantilla RFC (Request for Comments)

RFC: [Nombre del Sistema]

Autores: [Nombres de los integrantes]

- 1. Resumen Ejecutivo:** Descripción del problema del caso de estudio y la solución propuesta.
- 2. Atributos de Calidad:** Identificación de los 3 atributos críticos (ej. Disponibilidad, Latencia, Integridad).
- 3. Decisiones Arquitectónicas:** Justificación de la tecnología elegida (DBs, Broker, Lenguaje).
- 4. Trade-offs:** ¿Qué se sacrificó al elegir esta arquitectura? (ej. Consistencia eventual por alta disponibilidad).
- 5. Diagrama C4 Nivel 2:** Imagen o link al diagrama de contenedores.